

2 Solver comparison: multigrid against its alternatives

The geometric multigrid method of Section 1 is not the only way to solve the discrete system. This section places it against the standard alternatives for a symmetric positive definite operator, and shows that its advantage is not a favourable constant but a different asymptotic behaviour under mesh refinement.

2.1 What was compared

Five methods were applied to the *same* linear system: one torus, cubic Lagrange elements, one curved mapping, one right-hand side. Every method therefore reproduces the same discretisation error, and the comparison isolates the solver. Each run was performed in a separate process, and the multigrid level hierarchy is assembled only for the methods that use it, so no method is charged for machinery it does not touch. Two of the five are the same V-cycle: in Table 1, the multigrid rows differ only in whether a Krylov subspace steers the iteration.

Table 1: The five methods compared. All iterative methods stop at a relative residual of 10^{-10} .

Method	Description
CG + Jacobi	conjugate gradients, diagonal preconditioner
CG + SSOR	conjugate gradients, symmetric SOR preconditioner
MG standalone	the V-cycle iterated on the defect, no Krylov acceleration
CG + MG	the same V-cycle used as a preconditioner for CG
Direct	sparse LU factorisation (UMFPACK)

Two memory quantities are reported throughout, because they answer different questions. *Objects* counts the matrices, vectors and degree-of-freedom structures the solver holds directly. *Process* is the peak resident memory above a measured baseline. The difference between the two is memory allocated inside a library, which is precisely where a direct solver’s fill-in lives and where the first quantity sees nothing.

2.2 Results at the largest problem size

Table 2 gives one row per method at $N = 589,824$ unknowns (65,536 cells, seven uniform refinement cycles) — the largest problem the study could run, for the reason given below. *Linear algebra* is the preconditioner or factorisation together with its application, excluding assembly: that work is the same $\mathcal{O}(N)$ cost for every method and would otherwise dilute the comparison.

Table 2: All five methods at $N = 589,824$ unknowns. The linear-algebra column excludes assembly, which is common to all five. Memory is given both as solver objects and as process memory above baseline: the two answer different questions, and the gap between them is memory allocated inside a library.

Method	Iterations	Lin. alg. (s)	Total (s)	Objects (MB)	Process (MB)
CG + Jacobi	1000 [†]	179.1	309.2	203.5	430.5
CG + SSOR	957	692.6	835.6	203.5	451.7
MG standalone	31	114.3	335.6	443.5	833.0
CG + MG	15	62.1	329.7	443.5	793.1
Direct (UMFPACK)	1	652.0	800.4	203.5	3347.3

[†]Exhausted its budget of 1000 iterations without reaching tolerance.

Three observations follow.

Jacobi does not solve this problem at all. It exhausts its budget of 1000 iterations with the relative residual still at 9.7×10^{-8} , three orders of magnitude short of what every other method attains. Its H^1 error, 2.570×10^{-7} , is consequently *worse* than the other four, which agree with one another at 2.549×10^{-7} . This is the only point in the sweep at which the methods cease to produce the same answer, and it is why the benchmark records non-convergence as a result rather than treating it as a failure. Its row also appears inexpensive at 309 s only because it stopped early without solving the problem.

The direct solver requires 3.3 GB to factor a 204 MB matrix. That is a factor of 16.4. Its *object* memory is unremarkable — the same 203.5 MB held by Jacobi and SSOR, the active matrix and nothing more — but the *process* memory reveals what the first measure cannot: fill-in and its internal index arrays, allocated inside UMFPACK. The growth is not academic one level further on. Over the final refinement its memory rose by a factor of 4.0 for a factor of 4 in unknowns, so the next refinement projects beyond the 8 GB available on the machine used here. 589,824 unknowns is therefore a ceiling imposed by memory, not a stopping point chosen for convenience.

Multigrid is the fastest method as well as the most economical in the regime that matters. It is the only one of the five whose advantage is structural rather than a constant factor, which is the subject of the next subsection.

2.3 The scaling is the argument

A single problem size proves little, because a direct factorisation has a small constant and is competitive while the problem is small. What distinguishes the methods is how each responds to refinement. Table 3 reports the exponent b in $\log y = a + b \log N$, fitted over the largest half of the converged points, and Figure 1 shows the same data.

Table 3: Observed scaling exponents, fitted on the largest half of the converged points. A method with $b = 1$ for time and memory is optimal; larger b means the method degrades under refinement.

Method	Iterations	Linear algebra	Total time	Process memory
CG + Jacobi	$N^{0.50}$	$N^{1.26}$	$N^{1.08}$	$N^{0.51}$
CG + SSOR	$N^{0.47}$	$N^{1.40}$	$N^{1.28}$	$N^{0.66}$
MG standalone	$N^{0.00}$	$N^{0.89}$	$N^{0.94}$	$N^{0.74}$
CG + MG	$N^{0.02}$	$N^{0.90}$	$N^{0.99}$	$N^{0.74}$
Direct (UMFPACK)	$N^{0.00}$	$N^{1.40}$	$N^{1.26}$	$N^{0.93}$

Only multigrid has a mesh-independent iteration count. Jacobi requires 15 iterations at 144 unknowns and 673 at 147,456 — the count doubles with every refinement, $N^{0.50}$, which is the classical $\mathcal{O}(h^{-1})$ growth of an unpreconditioned Krylov method; by 589,824 unknowns it has ceased to converge within the budget at all. Multigrid’s count moves from 12 to 15 across the entire range, $N^{0.02}$. It is not that multigrid’s iterations are cheaper; it is that there are always approximately the same number of them. This is the property that makes the method valuable, and it is visible in the top-left panel of Figure 1 as a horizontal line where every other method climbs.

The consequence appears in the time exponent. Because Jacobi’s iteration count grows while each iteration costs $\mathcal{O}(N)$, its linear algebra grows like $N^{1.26}$; SSOR is worse at $N^{1.40}$. Multigrid grows like $N^{0.90}$. At the largest size this is the difference between 62 s and 693 s, and the gap widens without bound, because these are exponents and not constants.

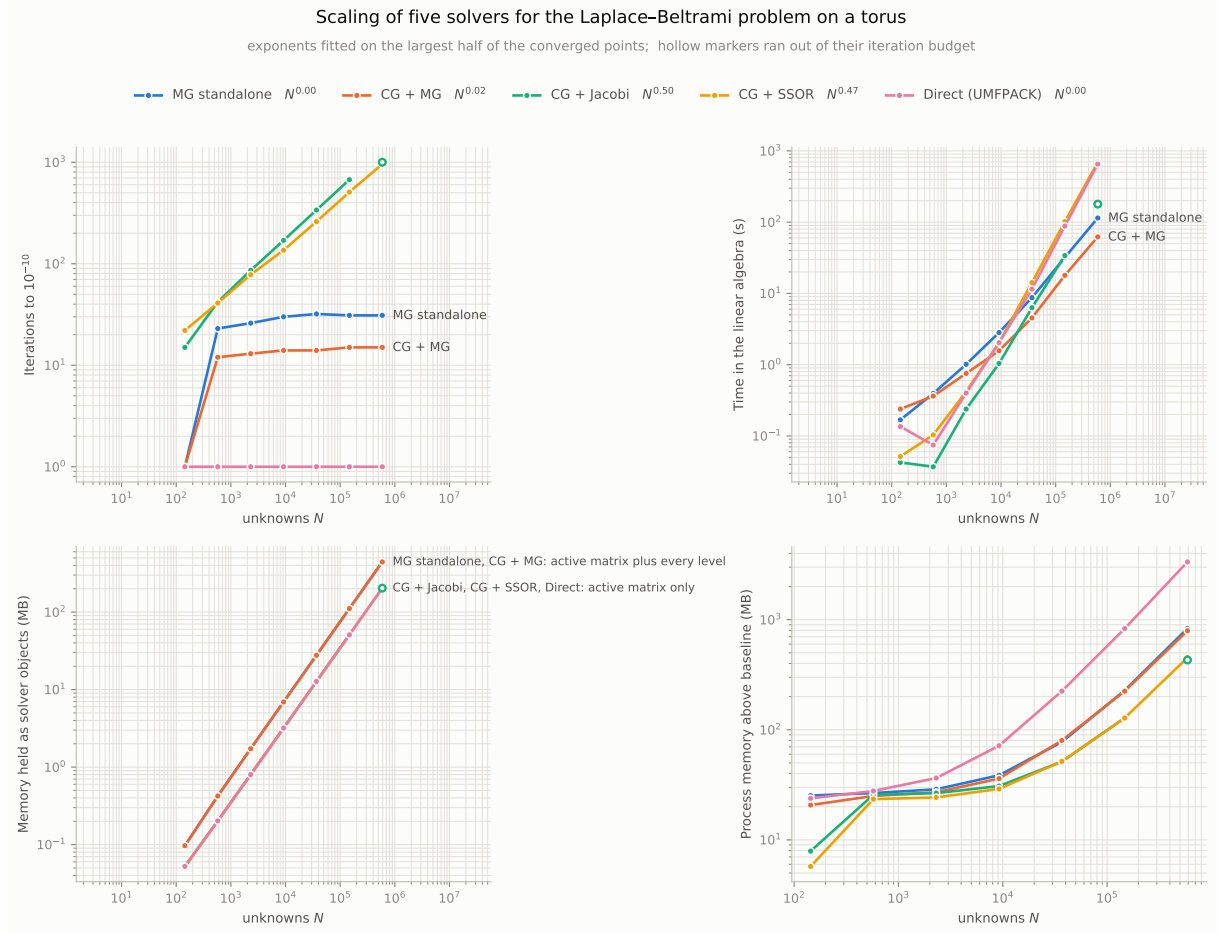


Figure 1: Scaling of the five methods from 144 to 589,824 unknowns, log-log in both axes, so a straight line is a power law and its slope is the exponent. Top left: iterations to reach tolerance. Top right: time in the linear algebra. Bottom left: memory held as solver objects. Bottom right: process memory above baseline. Hollow markers denote runs that exhausted their iteration budget and are therefore lower bounds; they are excluded from the fitted exponents.

Memory is where the direct solver is defeated, and the matrix does not show it. The direct method’s object memory tracks the others exactly; its process memory grows like $N^{0.93}$ against multigrid’s $N^{0.74}$, from a much larger constant. Multigrid does hold more than the simple preconditioners — 443.5 MB against 203.5 MB, because it stores an operator for every level — but this is a bounded factor, roughly 4/3 of the fine-grid matrix in two dimensions, and the two multigrid curves sit on lines parallel to the others rather than steeper. A constant factor is a price; a different slope is a ceiling.

2.4 Multigrid as a solver and as a preconditioner

The two multigrid variants are implemented on the same hierarchy and the same V-cycle, so they differ in exactly one respect: whether the iteration is plain defect correction or is accelerated by conjugate gradients. The result is worth stating carefully, because it is less dramatic than it is sometimes presented.

Both are mesh-independent, at $N^{0.00}$ and $N^{0.02}$ respectively. Standalone multigrid therefore already delivers the property that motivates the method; it does not require Krylov acceleration in order to be an optimal-order solver. What conjugate gradients contributes is a factor of approximately two: it requires 15 applications of the V-cycle where defect correction requires 31, and that ratio holds at every problem size in the sweep.

This is the expected outcome. Defect correction is a fixed-point iteration converging at the rate ρ of the V-cycle, whereas conjugate gradients minimises over a growing Krylov subspace and converges at a rate governed by $\sqrt{\kappa}$ instead. Conjugate gradients additionally guarantees monotone decrease of the error in the energy norm, which a bare V-cycle iteration does not, and it degrades gracefully if the smoother or the coarse-grid solve is imperfect. The honest summary is that CG + MG is about twice as fast, not asymptotically better.

2.5 Limitations of these measurements

The timings reported above were taken inside an `arm64` container running under emulation on an `x86_64` host, because that was the only available build of the finite element library. The iteration counts, the residuals, the discretisation errors and the memory in bytes are exact and independent of the architecture. *The wall-clock times are not.* The fitted exponents are ratios and largely survive the distortion, but the absolute times in Table 2 should not be quoted as hardware-representative; a native build would be required for figures intended for direct comparison with published timings.

Two further caveats apply. First, assembly is excluded from the linear-algebra column but not from the totals, and at the largest size it dominates — 162 s of multigrid’s 330 s total — so differences in total time understate the differences between the solvers themselves. Second, all results are for a single operator with a smooth variable coefficient, $\kappa(\xi, \eta) = 1.1 + \sin^2 \xi \cos^2 \eta$, bounded away from zero. Multigrid’s advantage is sensitive to that assumption; a coefficient with large jumps would require a different smoother and a different coarse-grid correction, and the exponents above would not be expected to carry over unchanged.